Discovery Executive Summary

Project: discovery-24jul-003 · **Generated:** 24/07/2026, 16:10:24

EXECUTIVE SUMMARY

This report consolidates the overall ratings, key findings, and recommended actions from the 1 discovery analysis run across this codebase (frontend and backend). Each section below reproduces that analysis's executive view; full evidence and diagrams live in the individual reports.

Portfolio Overview

#	Analysis	Overall Rating	Hotspot Score
1	Architecture & Design Analysis	HIGH RISK	—

1. Architecture & Design Analysis

OVERALL CODEBASE RATING — ARCHITECTURE & DESIGN

High Risk

Driven by High-Risk Missing Service Layer (H2), Missing Repository Pattern (H3), Shared Database Coupling (H9), and a fully Duplicated Parallel Backend (H10).

EXECUTIVE SUMMARY

FSDKC is a compact but structurally revealing "Klearcom" voice-observability platform spanning three layers: a PHP/Laravel 11 API (`backend/` — 6 controllers, 2 services, 4 Eloquent models), a React/TypeScript SPA (`frontend/` — 4 pages, 4 components, 1 hook, 1 Zustand store), and a full parallel Node/Express backend (`dev-api/` — 18 route handlers) that re-implements the entire server surface. Absolute file sizes are small, so no God Class (>1000 LOC) or oversized-component (>400 LOC) threshold is breached, but the design-level hotspots are severe: there is **no repository layer at all** (0 repository classes against 36 direct Eloquent access points, 25 of them inside controllers) and **no application-service tier for read paths**, so `DashboardController`, `ConnectController::checks`, and `LegacyReportController` carry KPI math, `extract()`-based mapping, and cross-domain queries directly. The dominant risk is **change amplification through duplication and shared data**: the reachability success-rate formula is copy-pasted across `ConnectController:72`, `RealTimeTestService:118`, `LegacyReportController:40`, and `dev-api/realtime.js:148`; `buildTree` exists verbatim in three files; and both the Laravel and Express backends read/write the *same* `klearcom` MongoDB collections with no ownership boundary. The Connect and Discovery bounded contexts declared in the module `AGENTS.md` files are violated

by `RealTimeTestService`, `LegacyReportController`, `DashboardController`, and the shared `MongoService`, all of which touch both domains at once. Layers covered: **backend** (13 PHP files), **frontend** (11 TS/TSX/JSX files), and a **second backend** (6 JS files). The overall verdict is **High Risk**, driven by the missing service/repository tiers (H2, H3), shared-database coupling (H9), and a fully duplicated parallel backend (H10).

1.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Rating
H1	Fat Controllers	Avg LOC per controller (+ business logic)	<150	150–300	>300	63 LOC avg, but 3/6 embed business logic	MODERATE
H2	Missing Service Layer	Controllers accessing repos/models	<10	10–20	>20	25 direct model access points	HIGH RISK
H3	Missing Repository Pattern	Direct DB/ORM access points	<10	10–20	>20	36 access points, 0 repositories	HIGH RISK
H4	Circular Dependencies	Dependency cycles	0	1–3	>3	0	GOOD
H5	Shared Utility Abuse	Utility files w/ business logic	0	1–5	>5	2 (`LegacyDataMapper` , `dev-api/store.js`)	MODERATE
H6	Direct SQL in Controllers	ORM/query-builder compliance %	>90%	60–90%	<60%	~85% (query builders in 2 controllers, no raw SQL)	MODERATE
H7	God Classes	Classes >1000 LOC	0	1–3	>3	0 (largest 231 LOC)	GOOD
H8	Domain Boundary Violations	Cross-domain access points	0	1–5	>5	4 (RealTimeTestService, LegacyReport, Dashboard, MongoService)	MODERATE
H9	Shared Database Coupling	Data stores shared across domains	<10%	10–30%	>30%	~37% (3/8 stores shared, 2 backends)	HIGH RISK

H10	Duplicated Parallel Backend *(additional)*	Business algorithms duplicated across backends	0	1–2	>2	3 (reachability ×4, buildTree ×3, KPI math ×2)	HIGH RISK
F1	Business Logic in Components	Avg LOC per component (worst-wins on max)	<150	150–300	>300	avg 79 LOC; largest ConnectPage 208 LOC w/ orchestration	MODERATE
F2	Missing Frontend Service/Data Layer	Components w/ inline API calls	<10	10–20	>20	7 files build endpoints inline (under threshold)	GOOD
F3	God / Oversized Components	Components >400 LOC	0	1–3	>3	0 (largest 208 LOC)	GOOD
F4	Prop Drilling / Global State Abuse	Max prop-drilling depth	≤2	3–4	>4	≤2 levels (Zustand + React Query)	GOOD
F5	Legacy / Inconsistent Component Patterns	Legacy-pattern components	0	1–10	>10	2 (class component w/ interval leak; no Error Boundary)	MODERATE

1.4 Actions Required

Hotspot	Action	Rating	Priority
H2 Missing Service Layer	Introduce `ConnectService` / `DiscoveryService` / `DashboardMetricsService`; move all controller model queries into them	HIGH RISK	CRITICAL
H3 Missing Repository Pattern	Add repository interfaces + Eloquent/Mongo impls bound in `AppServiceProvider`; encapsulate the "recent 20 checks" query	HIGH RISK	CRITICAL
H9 Shared Database Coupling	Give each context its own Mongo collections; add an ACL over the dev-api; adopt versioned collection contracts	HIGH RISK	HIGH
H10 Duplicated Parallel Backend	Consolidate reachability/ `buildTree` /KPI math into single shared services; add a cross-backend contract test	HIGH RISK	HIGH
H1 Fat Controllers	Extract KPI/reachability/mapping logic out of `Dashboard` / `Legacy` / `Connect` controllers into services	MODERATE	HIGH
H8 Domain Boundary Violations	Split `RealTimeTestService` into `Connect` / `Discovery` test services; consume cross-domain via published interfaces	MODERATE	HIGH

H5 Shared Utility Abuse	Replace `LegacyDataMapper` `extract()` with DTOs; extract `buildTree` from `store.js`	MODERATE	MEDIUM
H6 Direct SQL in Controllers	Move `LegacyReportController` / `ConnectController` query builders into repositories	MODERATE	MEDIUM
F1 Business Logic in Components	Move post-run invalidation into a `useConnectTest` hook; rebuild `LegacyDashboardWidget` on React Query	MODERATE	MEDIUM
F5 Legacy / Inconsistent Component Patterns	Convert `LegacyMonitorPoller` to a function component with cleanup; add an Error Boundary in `App.tsx`	MODERATE	MEDIUM

1.5 Expected Outcomes

- **Testable business rules:** reachability, IVR-tree, and KPI logic move into services/repositories that can be unit-tested without booting HTTP or a datastore, extending the coverage that today stops at `ReachabilityCalculationTest`.
- **Single source of truth:** consolidating the duplicated reachability (×4), `buildTree` (×3), and KPI (×2) algorithms (H1/H5/H10) turns a rule change into a one-file edit and eliminates dashboard-vs-report and prod-vs-dev divergence.
- **Independently evolvable domains:** enforcing the Connect and Discovery bounded contexts and giving each its own data ownership (H8/H9) makes either domain extractable into its own service without untangling `RealTimeTestService` or shared Mongo collections.
- **Swappable persistence:** a repository tier bound in `AppServiceProvider` (H3) decouples business logic from Eloquent/Mongo, enabling schema changes and datastore swaps behind stable interfaces.
- **Resilient frontend:** extracting orchestration into hooks and adding a top-level Error Boundary (F1/F5) stops the interval leak in `LegacyMonitorPoller` and prevents a single failed fetch from crashing the entire SPA.